# CAA Test 1

```
enum class State {
  IDLE, LOCK, FILL, WASH, DRAIN, UNLOCK, BUZZ, ERROR_STATE
};

enum class Mode {
  QUICK = 2, NORMAL = 4, HEAVY = 6
};

struct Inputs {
  bool start = false;
  bool door_closed = true;
  bool water_ok = false;
  bool overload = false;
};

struct Outputs {
  bool motor = false;
  bool valve = false;
  bool door_lock = false;
  bool buzzer = false;
};

string stateName(State s) {
  switch (s) {
  case State::IDLE: return "IDLE";
  case State::LOCK: return "LOCK";
  case State::FILL: return "FILL";
  case State::WASH: return "WASH";
  case State::DRAIN: return "DRAIN";
  case State::UNLOCK: return "UNLOCK";
  case State::BUZZ: return "BUZZ";
  case State::ERROR_STATE: return "ERROR";
  }
  return "UNKNOWN";
}

Outputs getOutputs(State s) {
  switch (s) {
```

```
  case State::LOCK: return {false, false, true, false};
  case State::FILL: return {false, true, true, false};
  case State::WASH: return {true, false, true, false};
  case State::DRAIN: return {false, false, true, false};
  case State::UNLOCK: return {false, false, false, false};
  case State::BUZZ: return {false, false, false, true};
  case State::ERROR_STATE: return {false, false, true, true};
  default: return {false, false, false, false};
  }
}

class WashingMachineController {
public:
  explicit WashingMachineController(Mode mode)
      : currentState(State::IDLE), mode(mode), stateTimer(0),
        washCycles(0), totalCycles(static_cast<int>(mode)) {}

  bool tick(int cycleNum, const Inputs& inputs) {
    if (checkInterrupt(inputs)) {
      currentState = State::ERROR_STATE;
    }

    Outputs out = getOutputs(currentState);

    if (currentState == State::ERROR_STATE) {
      return false;
    }

    if (currentState == State::BUZZ) {
      return false;
    }

    transitionState(inputs);
    return true;
  }

  State getState() const { return currentState; }

private:
  State currentState;
  Mode mode;
  int stateTimer;
  int washCycles;
```

```
int totalCycles;

bool checkInterrupt(const Inputs& in) {
  if (currentState == State::ERROR_STATE) return false;
  if (currentState == State::IDLE) return false;
  if (currentState == State::BUZZ) return false;

  if (in.overload) return true;
  if (!in.door_closed) return true;

  return false;
}

void transitionState(const Inputs& in) {
  switch (currentState) {
  case State::IDLE:
    if (in.start && in.door_closed)
      currentState = State::LOCK;
    break;

  case State::LOCK:
    stateTimer++;
    if (stateTimer >= 2) {
      stateTimer = 0;
      currentState = State::FILL;
    }
    break;

  case State::FILL:
    if (in.water_ok)
      currentState = State::WASH;
    break;

  case State::WASH:
    washCycles++;
    if (washCycles >= totalCycles) {
      washCycles = 0;
      currentState = State::DRAIN;
    }
    break;

  case State::DRAIN:
    currentState = State::UNLOCK;
```

```
      break;

    case State::UNLOCK:
      stateTimer++;
      if (stateTimer >= 2) {
        stateTimer = 0;
        currentState = State::BUZZ;
      }
      break;

    case State::BUZZ:
      break;

    case State::ERROR_STATE:
      break;
    }
  }
};
```